

“ON-THE-FLY PLAN GRAPH CONSTRUCTION”

章节审稿式诊断与系统重写方案

基于 bare_jrnl_new_sample4(2).pdf 第 7–10 页

审查目标：检查本节的段落推进、信息重复、符号一致性、术语规范性和算法逻辑，并给出一套可直接替换正文的英文版本。报告同时区分了“纯写作问题”和“会被严格审稿人视为正确性缺口的问题”。

结论等级： **Major Revision**

生成日期：2026 年 7 月 15 日

目录

1 审稿结论与最重要的修改方向	2
2 逐项问题：严格审稿人会如何质疑	2
2.1 章节开头重复且没有立即突出“本文到底改了些什么”	2
2.2 “Bstate”不是规范术语，且 expansion order 的描述互相矛盾	3
2.3 score 在算法中出现得太早，计算顺序不成立	3
2.4 共享图允许多个父节点，但 task 和 assignment 却存放在节点中	4
2.5 “当前 cost 更小”不能支配机器人位置不同的执行状态	4
2.6 prefix/suffix 标志不能确定 suffix cycle 的闭合条件	4
2.7 贪心分配失败不等于 subtask 不可行	5
2.8 self-loop condition 在 subtask 定义中出现，却没有进入可执行性计算	5
2.9 remaining-action heuristic 的定义不完整且名称过强	5
2.10 后处理剪枝充满未定义的实现术语	5
3 符号和术语统一表	6
4 推荐的章节重组方式	7
5 推荐的数学数据结构：为什么需要“两层表示”	8
5.1 逻辑计划图节点	8
5.2 路径相关 execution label	8
5.3 边记录 transition-specific 信息	8
5.4 允许删除 label 的支配条件	8
6 可直接替换的英文正文：推荐版本	9
7 对 Algorithm 2 和 Algorithm 3 的具体修改清单	14
7.1 Algorithm 2 必须修改的行	14
7.2 Algorithm 3 必须修改的行	14
8 可直接替换的关键句子	15
9 必须同步修改的其他章节	15
9.1 Definition 5: Subtask	15
9.2 Section VII: Branch-and-Bound Optimization	15
9.3 Complexity Analysis	16
9.4 Theoretical Analysis	16
10 投稿前核查清单	16
11 最终评价	17

1 审稿结论与最重要的修改方向

本节目前不适合只做句子级润色。真正的问题不是个别英文不够自然，而是“NBA 状态生成顺序、共享计划图、路径相关机器人状态、贪心分配、启发式评分和后处理剪枝”被混写在同一套叙述中。由此产生了多处定义与算法不一致，部分问题还会直接影响完整性和最优性结论。

从严格审稿人的视角，本节有一个值得保留的清晰主线：在 GBA-to-NBA 退广义化过程中，每生成一条 NBA transition，就立即生成对应 subtask、检查机器人分配、扩展计划结构，并在首次闭合 prefix-suffix 结构时返回可执行解。当前版本的问题是，这条主线被以下内容遮蔽：

1. 前三段反复说明“同步构造 NBA 和 plan graph、无需等待完整 NBA、可以提前返回解”；
2. plan node 同时承担“逻辑节点”和“某一条具体执行路径的机器人状态”，但又允许多个父节点共享，语义不自洽；
3. Algorithm 2 表面上使用全局最小 score 的队列，而 Traversal Rules 又声明 score 只改变局部 DFS 顺序；
4. score 在进行机器人分配之前被用于排序，但 score 本身依赖具体 plan node 的位置、完成时间和已完成命题；
5. 若贪心分配失败就直接删除 successor，会把“贪心没找到”误写成“问题不可行”，从而破坏完整性；
6. suffix 阶段只用 pre/suf/dead 表示，不能记录本次 suffix cycle 要返回的具体 accepting state；
7. 后处理部分使用了 live corridor、hard-dead state、internal cut mark、sibling coverage 等未定义的实现术语。

推荐的核心改法：把“计划图拓扑”和“路径相关执行状态”分开。

- 计划图节点只记录 NBA progress 和 prefix-suffix phase；
- NBA transition 对应的 subtask 放在计划图边上；
- 每个计划图节点维护一组 execution labels，每个 label 记录机器人位置、完成时间和评分所需的已完成 action propositions；
- 工作表中扩展的是 execution label，而不是只扩展 NBA state 一次；
- score 只用于排列可行 successor 的访问顺序，不参与删除；
- 只有严格的不可行性检验或有证明的支配关系才允许删除 label。

这套表述既符合共享有向图的语义，也避免继续使用 anchor、runtime payload 和 late-anchor recovery 等自造术语。

2 逐项问题：严格审稿人会如何质疑

2.1 章节开头重复且没有立即突出“本文到底改了什么”

当前开头先介绍 LTL2BA 的 DFS，再介绍 score 排序，随后重新定义 pruned GBA 和 NBA state，最后再次说明 NBA 与 plan graph 同步构造、可以提前返回解。四段中至少有两段表达的是同一件事。

审稿人可能的评价：本节没有在第一段清楚区分“继承自 LTL2BA 的部分”和“本文新增的部分”。读者读完一页后才勉强拼出三个新增操作：即时分配、score 排序、共享计划图。

修改方法：开头压缩为两段。

- 第一段只交代标准流程：GBA-to-NBA 退广义化按需生成 $q_B = (q_g, \ell)$ 及其 outgoing transitions。
- 第二段明确本文的三项改动：每条新 transition 立即做可执行性扩展；只对可行 successors 排序；首次闭合 prefix-suffix cycle 时立即提取方案。随后直接进入数据结构定义。

不要再次完整写出 $G_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, F_g)$ ，因为该符号已在前文定义。

2.2 “Bstate”不是规范术语，且 expansion order 的描述互相矛盾

正文和伪代码反复使用 *Bstate*。该词并不是领域内常见术语，参考论文通常直接写 *NBA state* 或用符号 q_B 。更严重的是：

- Algorithm 2 第 5 行写成“从队列中弹出 residual-obligation score 最小的 Bstate”，这表示全局 priority queue；
- Section VI-C 又明确说 score 只改变当前 pending batch 的局部顺序，“rather than defining a global priority queue”。

这不是措辞差异，而是两种不同算法。

修改方法：必须根据真实实现二选一。

1. 若实现沿用 LTL2BA 的 DFS，只对子节点排序，则统一写成 *score-guided depth-first expansion*，工作表使用 stack；候选 successors 按 score 排序后逆序压栈。
2. 若实现确实每次弹出全局最小 score，则使用 priority queue，并删除所有“local pending batch / not a global priority queue”的表述。

本报告后面的重写版本采用第一种，即“局部 score-guided DFS”，因为它与当前 Traversal Rules 的自述更一致。

2.3 score 在算法中出现得太早，计算顺序不成立

当前 Algorithm 2 先对 candidate successor NBA states 进行 score 排序，然后才遍历 attached plan-node anchors 并调用 Algorithm 3 做分配。然而式 (6)–(8) 中的 score 依赖 J_i 、 X_i 、 Θ_i 和路径上已满足的 positive propositions。换言之，在尚未选择具体 parent plan context、尚未完成分配时，score 无法计算。

修改方法：顺序应改为：

生成 NBA transition → 对每个 execution label 做可行分配
 → 得到 successor execution state
 → 计算 score 并排序。

score 必须绑定到“具体 parent execution label + transition”或“具体 successor execution label”，而不能无条件绑定到一个 NBA state。

2.4 共享图允许多个父节点，但 task 和 assignment 却存放在节点中

当前 Definition 8 将 T_i 和 C_i 放在 plan node ν_i 内，同时又允许 ν_i 有多个 structural parents。若两个父节点通过不同 NBA transitions 到达同一个共享节点，那么 incoming subtask 和 assignment 很可能不同。此时单一的 T_i 和 C_i 无法同时表示两条入边。

这是结构性错误。在树结构中，像 Chen and Kan 的 PDT 那样把当前 subtask 和 assignment 放在节点里是成立的，因为每个节点只有一个父节点；在允许多父节点的共享图中，transition-specific 信息应放在边上。

修改方法：

- plan node 记录未来扩展真正需要的逻辑状态；
- plan-graph edge 记录 induced subtask 和本次分配；
- path-dependent robot positions/times 不应与共享结构节点混为一个对象，而应作为该节点上的 execution label。

2.5 “当前 cost 更小”不能支配机器人位置不同的执行状态

当前文本规定：若新连接带来的 realized cost 更小，就替换 shared node 的 representative predecessor、position vector 和 completion-time vector，并向 descendants 传播。该规则没有成立所需的支配条件。

例如，同一 NBA state 下有两个执行状态：

$$\begin{aligned} J_1 &= 8, 0 \quad x_1(r) = l_1, 0 \quad d(l_1, l_3) = 10, \\ J_2 &= 9, 0 \quad x_2(r) = l_3. \end{aligned}$$

若下一任务位于 l_3 ，第一状态最早约在 18 时刻完成，而第二状态可在 9 时刻继续。虽然 $J_1 < J_2$ ，但第一状态并不支配第二状态。

修改方法：至少使用如下安全条件：两个 execution labels 必须位于同一逻辑节点、具有相同机器人位置和相同评分历史，并且一个 label 的所有机器人完成时间均不晚于另一个：

$$X^1 = X^2, 0 \quad D^1 = D^2, 0 \quad \theta^1(r_k) \leq \theta^2(r_k), \forall r_k \in \mathcal{R}.$$

仅比较 makespan $\max_k \theta(r_k)$ 不足以构成支配关系。若论文不准备给出这类条件与证明，应删除“representative payload update + descendant propagation”，只在执行状态完全相同时共享节点。

2.6 prefix/suffix 标志不能确定 suffix cycle 的闭合条件

当前 $\text{Prog}_i \in \{\text{pre}, \text{suf}, \text{dead}\}$ 只能说明“进入了 suffix”，却没有记录 suffix 从哪个 accepting NBA state 开始。对于存在多个 accepting states 的 NBA，仅再次到达任意 accepting state 并不等于闭合了选定的 suffix cycle。

修改方法：phase 字段应为

$$\kappa_i \in \{\text{pre}\} \cup Q_{B,F}.$$

当尚未进入 suffix 时， $\kappa_i = \text{pre}$ ；第一次到达 accepting state q_f 后，将 $\kappa_i = q_f$ ，即把 q_f 作为本次 suffix root。只有当父 label 已处于 $\kappa_i = q_f$ 且 successor 再次到达 q_f 时，才完成一次 suffix cycle。

dead 不是 planning stage, 而且正文又说 temporary dead child 不保留, 因此应从节点定义中删除。

2.7 贪心分配失败不等于 subtask 不可行

当前方法对每个 action proposition 选择预计到达时间最小的机器人, 并禁止同一机器人在同一同步任务中重复使用。若按固定顺序贪心选择, 可能出现假不可行。

考虑两个机器人和两个单机器人 action: 机器人 r_1 可到达 a_1, a_2 , 机器人 r_2 只能到达 a_1 。若先处理 a_1 并因到达更早选择 r_1 , 则 a_2 无人可选; 但 $r_2 \rightarrow a_1, r_1 \rightarrow a_2$ 是可行分配。

修改方法: 保留贪心作为快速候选生成器, 但在删除 successor 之前, 必须通过二分图 b -matching 或等价的精确可行性检验确认不存在互斥分配。否则, Theoretical Analysis 中“若 feasible solution 存在则不会被删除”的证明不成立。

2.8 self-loop condition 在 subtask 定义中出现, 却没有进入可执行性计算

前文将 subtask 定义为 transition condition 与 source-state self-loop condition 的二元组, 但本节的到达时间和分配只处理 transition 中的 positive action propositions。正文没有说明机器人移动和等待期间如何维持 self-loop condition, 尤其是负命题、区域约束或持续保持的 action proposition。

修改方法: 明确下列二者之一:

- $d_k(x, l)$ 是在满足 source self-loop guard 的可行路径集合上计算的最短距离, 不存在此类路径时取 $+\infty$; 或
- 高层分配之后调用独立 motion-feasibility oracle 验证 holding condition, 验证失败则不接受 execution label。

若两者都没有实现, 则不应把当前 plan 称为“executable”。

2.9 remaining-action heuristic 的定义不完整且名称过强

当前 *must-eventually set* 的递归规则遗漏了主语法中的 until operator; 同时把 $\Box\psi$ 和 $\Diamond\psi$ 简单设为同一规则, 却没有说明 repeated/persistent obligation 如何处理。 $A^+(\nu_i, \tau)$ 也未包含在 plan node 定义中。

修改方法:

- 改名为 *required-action approximation* 或更普通的 *remaining-action heuristic*;
- 在 NNF 的核心语法上给出完整、紧凑的递归式, 包括 U 和 NNF 转换后可能出现的 release R ;
- 明确这只是排序启发式, 不是 admissible lower bound, 不参与 branch pruning, 也不支撑最优性证明;
- 若要减去已完成 action propositions, 必须把相应集合明确存入 execution label。

2.10 后处理剪枝充满未定义的实现术语

live corridor、*hard-dead automaton state*、*internal cut mark*、*local signature*、*covers its successor structure* 都没有数学定义或伪代码。严格审稿人无法判断这些操作是否保持可行解。

修改方法: 主文只保留两个可验证的操作:

1. 从 root 做 forward reachability;
2. 从 terminal plan nodes 做 reverse reachability;
3. 只保留两者交集集中的节点和边。

若保留 dominance pruning, 使用明确的 componentwise dominance 条件并给出 lemma; 不要用 “sibling covers another successor structure” 这类难以验证的描述。实现中的 cut flags 可放入附录或代码说明, 不宜进入核心方法正文。

3 符号和术语统一表

当前写法	建议统一写法	理由与具体处理
Bstate	NBA state, 符号 q_B	非标准缩写, 全文删除。
T_i, T_{ij}, T_j, T	ω_i, ω_{ij}	前文 Definition 5 已用 ω 表示 subtask; 不要另起一套 T , 也避免与完成时间 $T(\Pi)$ 冲突。
候选 transition 记为 τ	e_{ij}^B	$\tau(r_k)$ 已用于机器人类型映射, 继续用 τ 表示 transition 会造成冲突。
σ 同时表示 symbol 和 symbolic condition	transition guard 用 γ , 具体 valuation 用 σ	与前一节 GBA pruning 的 guard/symbol 区分保持一致。若全文暂不改 guard 记号, 至少要说明 σ 在该处是 Boolean guard, 而非单一 alphabet symbol。
$\text{Prog}_i \in \{\text{pre}, \text{suf}, \text{dead}\}$	$\kappa_i \in \{\text{pre}\} \cup Q_{B,F}$	记录具体 suffix root; 删除无意义的 dead stage。
plan-node anchor	plan node / execution label	<i>anchor</i> 无定义且与实际数据结构混淆。
runtime payload	execution state (X, Θ) 或 execution label	软件实现词, 不适合作为核心数学术语。
structural edge	plan-graph edge	若边上存 subtask/assignment, 可直接称 edge。
representative predecessor	predecessor pointer used for plan extraction	只有在精确定义 label 和更新规则后才能使用; 不要把它与图的全部父边混为一谈。
late-anchor recovery	re-expansion of a newly inserted execution label	不需要创造新名词。工作表扩展 label 后, 所谓 “late recovery” 自然消失。
residual-obligation pressure	remaining-action score	前者名词堆叠且容易被误认为理论 lower bound。
must-eventually set	required-action approximation $\mathcal{M}$	明确只是句法启发式, 不夸大其语义保证。
$R(\nu_i, \tau)$	$\mathcal{U}_i$ 或 $\mathcal{U}(\lambda_i)$	避免与机器人集合 $\mathcal{R}$ 混淆, 也避免 τ 冲突。
J_i	定义为 $J(\lambda_i) = \max_{r_k \in \mathcal{R}} \theta_i(r_k)$	当前式 (7)–(8) 直接使用但没有定义。

当前写法	建议统一写法	理由与具体处理
LB_i	本节删除	Definition 8 的 tuple 中没有 LB_i ，且本节 score 不是 BnB lower bound。真正的 LB 留在 Section VII。
$A^+(\nu_i, \tau)$	execution label 中的 $\mathcal{D}_i$ 并明确更新	当前符号凭空出现，且多父节点下路径历史不唯一。
$\mathcal{Q}_B$ 与 Q_B	Q_B 仅表示 NBA state set, 工作表写 OPEN	视觉上极易混淆。
$T_\emptyset, C_\emptyset$	$\emptyset$	未定义的空对象记号没有必要。
positive/ negative	positive and negative	删除斜杠前后不一致的空格。
live corridor	root-to-terminal subgraph	使用标准图论表述。
hard-dead / cut mark	从主文删除或在附录正式定义	当前属于代码实现细节，无法支撑理论结论。

4 推荐的章节重组方式

建议把当前 Section VI 从“五个子节 + 两段重复 child generation”重组为四个子节：

1. **Plan Graph and Execution Labels**: 只定义共享逻辑图、phase 和 path-dependent labels;
2. **Feasible Label Extension**: 集中说明 subtask、分配、同步时间与状态更新;
3. **Score-Guided On-the-Fly Expansion**: 先定义 score，再给 Algorithm 2 和 Algorithm 3;
4. **Graph Simplification and Plan Extraction**: 只保留标准 reachability pruning 和 predecessor-chain extraction。

当前内容	建议处理
Section VI 开头三段	合并为两段，删除 GBA tuple 重复定义。
A. Plan Graph and Planning Node	改成“结构节点 + execution label”，task/assignment 移到边。
B. Construction of Plan Graph 首段	与新的 A 合并，不再重复 root initialization。
B. Construction of Plan Graph 后半 + B.1	合并成“Feasible Label Extension”，到达时间公式只保留一次。
Algorithm 3	改名为函数 ExtendLabel，不再写“Algorithm 3(...)”。
B.2 + C. Traversal Rules	合并为“Score-Guided Expansion”；先产生可行 successor，再计算 score。
D. Final Post-Construction Pruning	只保留 forward/backward reachability；删除未定义实现标记。
E. Plan Extraction	改为沿 predecessor label pointers 读取边上的 subtask/assignment。

5 推荐的数学数据结构：为什么需要“两层表示”

5.1 逻辑计划图节点

计划图节点只表示对未来扩展有共同语义的逻辑状态：

$$v_i = (q_{B,i}, \kappa_i), \quad \kappa_i \in \{\text{pre}\} \cup Q_{B,F}.$$

若 $\kappa_i = \text{pre}$ ，当前仍在 prefix；若 $\kappa_i = q_f \in Q_{B,F}$ ，则 q_f 是已经选定的 suffix root。这样可以在不额外增加“suf/dead”名词的情况下，准确判断 suffix 是否闭合。

5.2 路径相关 execution label

同一逻辑节点可能由不同机器人分配到达，因此每个节点维护一组 labels：

$$\lambda_i = (X_i, \Theta_i, \mathcal{D}_i),$$

其中 X_i 和 Θ_i 沿用前文的机器人位置与完成时间向量； $\mathcal{D}_i \subseteq \mathcal{A}$ 只用于 remaining-action score，表示沿该 label 的 predecessor path 至少完成过一次的 positive action propositions。每个 label 在实现中再保存一个 predecessor label pointer 和 incoming plan edge，用于提取第一条方案；这两个指针不必进入数学 tuple。

5.3 边记录 transition-specific 信息

从 v_i 到 v_j 的 plan-graph edge 对应一条 NBA transition，并保存：

$$\omega_{ij} = (\gamma_{ij}, \gamma_i^s), \quad C_{ij} : \mathcal{A}(\omega_{ij}) \rightarrow 2^{\mathcal{R}}.$$

这样，即使一个逻辑节点有多条入边，每条入边也可以保留自己的 subtask 和 assignment，不会出现单一 T_i, C_i 无法解释多父节点的问题。后续 BnB 也应改成“读取边上的 subtask”，而不是“读取 child node 上的 subtask”。

5.4 允许删除 label 的支配条件

对同一逻辑节点 v 上的两个 labels，只有在

$$X^1 = X^2, \quad \mathcal{D}^1 = \mathcal{D}^2, \quad \theta^1(r_k) \leq \theta^2(r_k), \quad \forall r_k \in \mathcal{R}$$

时，才可认为 λ^1 支配 λ^2 。这一条件体现的是：从相同逻辑状态和相同机器人位置出发，所有机器人都不晚于另一 label，后续任一同样的分配都不会更差。仅有 $J(\lambda^1) < J(\lambda^2)$ 远远不够。

若不采用“两层表示”，还有一个较小改动的备选方案：彻底取消多父节点共享，把结构改成 planning tree。此时可以像参考文献中的 PDT 一样把 subtask、assignment、position 和 time 都存入节点。不能一边保留树节点的数据结构，一边宣称节点可以由不同父路径共享。

6 可直接替换的英文正文：推荐版本

可直接用于论文的修改文本

使用前提。 下面的完整替换文本采用四项必要修正：

- (1) plan graph 使用“逻辑节点 + execution labels”；
 - (2) subtask 和 assignment 存在边上；
 - (3) greedy failure 在删除前由精确 matching 检查；
 - (4) score 只改变访问顺序，不参与 pruning。
- 若代码并不满足这些条件，不能原样使用相应的 completeness 表述。

VI. ON-THE-FLY NBA AND PLAN-GRAPH CONSTRUCTION

After the GBA-level pruning, the pruned GBA is degeneralized into an NBA following the standard LTL2BA construction. An NBA state is generated on demand as $q_B = (q_g, \ell)$, where q_g is a GBA state and ℓ is the acceptance-progress index defined in Section V. The original LTL2BA implementation explores newly generated states in depth-first order according to their generation order. We retain this on-the-fly construction but evaluate each generated NBA transition immediately at the task-allocation level. Feasible successor execution labels are locally ordered by a planning score before they are pushed onto the depth-first stack. The score changes only the exploration order; it does not remove NBA transitions or plan-graph branches.

The NBA and the plan graph are therefore constructed together. Whenever an NBA transition is generated, its induced subtask is used to extend the currently available execution labels. A label is discarded only if the induced synchronized assignment is infeasible or if it is dominated by another label under the condition stated below. Once an execution label closes an accepting prefix-suffix structure, a feasible plan is extracted immediately, without waiting for the complete NBA and plan graph to be generated.

A. Plan Graph and Execution Labels

We separate the logical plan-graph structure from path-dependent robot execution states. This distinction is necessary because different partial assignments may reach the same NBA state with different robot positions and completion times.

Let $G_P = (V_P, E_P)$ be the directed plan graph. Each plan node $v_i \in V_P$ is defined by

$$v_i = (q_{B,i}, \kappa_i),$$

where $q_{B,i} \in Q_B$ is the associated NBA state and $\kappa_i \in \{\text{pre}\} \cup Q_{B,F}$ records the prefix-suffix phase. The value $\kappa_i = \text{pre}$ indicates that no suffix root has been selected. When an accepting state $q_f \in Q_{B,F}$ is first reached from the prefix, κ_i is set to q_f , which is then retained as the suffix root. A suffix cycle is completed only when a later transition returns to this same state q_f .

Each plan-graph edge corresponds to an NBA transition $e_{ij}^B = (q_{B,i}, \gamma_{ij}, q_{B,j})$ and stores the induced subtask

$$\omega_{ij} = (\gamma_{ij}, \gamma_i^s),$$

where γ_i^s is the self-loop guard at $q_{B,i}$, together with the robot assignment selected for this subtask. Placing the subtask and assignment on the edge avoids ambiguity when a plan node has multiple predeces-

sors.

A plan node may carry multiple execution labels. An execution label attached to v_i is

$$\lambda_i = (X_i, \Theta_i, \mathcal{D}_i),$$

where $X_i = (x_i(r_1), \dots, x_i(r_{n_r}))$ and $\Theta_i = (\theta_i(r_1), \dots, \theta_i(r_{n_r}))$ are the predicted robot positions and completion times, respectively. The auxiliary set $\mathcal{D}_i \subseteq \mathcal{A}$ contains the positive action propositions that have been completed at least once along the predecessor path and is used only by the ordering heuristic in Section VI-C. The realized cost of λ_i is

$$J(\lambda_i) = \max_{r_k \in \mathcal{R}} \theta_i(r_k).$$

The root node is associated with an initial NBA state and $\kappa_0 = \text{pre}$. Its initial label is $\lambda_0 = (X_0, \mathbf{0}, \emptyset)$, where X_0 contains the measured initial robot positions. If the NBA has multiple initial states, a virtual root is connected to one root plan node for each initial state.

For labels attached to the same plan node, λ^1 dominates λ^2 only if they have the same position vector and completed-action set and satisfy

$$\theta^1(r_k) \leq \theta^2(r_k), \quad \forall r_k \in \mathcal{R}.$$

This componentwise condition is required because a smaller current makespan alone does not imply a better future execution state when robot positions or individual completion times differ.

B. Feasible Label Extension

Consider an execution label λ_i attached to $v_i = (q_{B,i}, \kappa_i)$ and an outgoing NBA transition $e_{ij}^B = (q_{B,i}, \gamma_{ij}, q_{B,j})$. Let $\mathcal{A}(\omega_{ij})$ be the set of positive action propositions that must be satisfied to execute the induced subtask. For each $a \in \mathcal{A}(\omega_{ij})$, recall that

$$c(a) = (\tau_a, n_a, l_a),$$

where τ_a , n_a , and l_a are the required robot type, cardinality, and target region, respectively. For each type-compatible robot $r_k \in \mathcal{R}_{\tau_a}$, its predicted arrival time is

$$\hat{t}_i(r_k, a) = \theta_i(r_k) + \frac{d_k(x_i(r_k), l_a)}{v(r_k)}.$$

Here, d_k is evaluated over paths that preserve the self-loop guard γ_i^s ; it is set to $+\infty$ when no such path exists. The assignment C_{ij} must select n_a robots for every required action proposition and must satisfy

$$C_{ij}(a) \subseteq \mathcal{R}_{\tau_a}, \quad |C_{ij}(a)| = n_a, \quad C_{ij}(a) \cap C_{ij}(a') = \emptyset$$

for distinct action propositions a and a' in the same synchronized subtask. The implementation first applies the greedy earliest-arrival rule to obtain a candidate assignment. If the greedy rule fails, infeasibility is confirmed by a bipartite b -matching test before the successor is discarded. This fallback prevents a feasible transition from being removed solely because of the order in which the greedy rule processes action propositions.

For a feasible assignment, all actions in the transition guard are synchronized at

$$t_{ij} = \max_{a \in \mathcal{A}(\omega_{ij})} \max_{r_k \in C_{ij}(a)} \widehat{t}_i(r_k, a).$$

Each selected robot is placed at its assigned target region and its completion time is set to t_{ij} ; unselected robots retain their parent positions and completion times. If the transition contains no positive action proposition, $X_j = X_i$ and $\Theta_j = \Theta_i$. The completed-action set is updated as

$$\mathcal{D}_j = \mathcal{D}_i \cup \mathcal{A}^+(\gamma_{ij}),$$

where $\mathcal{A}^+(\gamma_{ij})$ denotes the positive action propositions asserted by the transition guard.

The phase is updated by

$$\kappa_j = \begin{cases} q_{B,j}, & \kappa_i = \text{pre and } q_{B,j} \in Q_{B,F}, \\ \kappa_i, & \text{otherwise.} \end{cases}$$

The successor completes a prefix-suffix structure if $\kappa_i = q_f \in Q_{B,F}$ and $q_{B,j} = q_f$. In this case, the transition just generated closes one traversal of the selected suffix cycle and the successor label is recorded as terminal.

C. Score-Guided On-the-Fly Expansion

The expansion order is guided by a lightweight estimate of the positive actions that may still need to be completed. Let $\text{Req}(\psi) \subseteq \mathcal{A}$ be the required-action approximation of a subformula ψ in negative normal form. It is computed by

$$\text{Req}(\top) = \text{Req}(\perp) = \text{Req}(\neg a) = \emptyset, \quad \text{Req}(a) = \{a\},$$

$$\text{Req}(\psi_1 \wedge \psi_2) = \text{Req}(\psi_1) \cup \text{Req}(\psi_2), \quad \text{Req}(\psi_1 \vee \psi_2) = \text{Req}(\psi_1) \cap \text{Req}(\psi_2),$$

$$\text{Req}(\bigcirc \psi) = \text{Req}(\psi), \quad \text{Req}(\psi_1 \mathbf{U} \psi_2) = \text{Req}(\psi_2), \quad \text{Req}(\psi_1 \mathbf{R} \psi_2) = \text{Req}(\psi_2),$$

where $\mathbf{R}$ is the release operator introduced by the negative-normal-form transformation. Let $\mathcal{M} = \text{Req}(\varphi)$. For a newly generated label λ_j , define

$$\mathcal{U}(\lambda_j) = \mathcal{M} \setminus \mathcal{D}_j.$$

For each $a \in \mathcal{U}(\lambda_j)$, let $h_a(\lambda_j)$ be the n_a -th smallest arrival time among the robots in $\mathcal{R}_{\tau_a}$, computed from X_j and Θ_j . The successor score is

$$S(\lambda_j) = \max \left\{ J(\lambda_j), \max_{a \in \mathcal{U}(\lambda_j)} h_a(\lambda_j) \right\},$$

with the second maximum defined as $J(\lambda_j)$ when $\mathcal{U}(\lambda_j) = \emptyset$. This score is a syntactic ordering heuristic rather than an admissible lower bound. In particular, it does not fully represent repeated or persistent obligations. It is therefore used only to order feasible successors and never to prune a branch or certify optimality.

Algorithm 2 summarizes the joint construction. The work list contains unexpanded execution labels

rather than NBA states. When a label is expanded, outgoing NBA transitions of its associated NBA state are generated on demand and cached. Thus, if a new execution label reaches an NBA state whose automaton transitions were generated earlier, the cached transitions are simply applied to the new label; no separate “late-anchor recovery” mechanism is required. Feasible successor labels are sorted by nondecreasing score and pushed onto the depth-first stack in reverse order. Ties are resolved by the original LTL2BA generation order.

Algorithm 2 On-the-fly NBA and plan-graph construction

Require: Pruned GBA G_φ^- , robot team $\mathcal{R}$, initial state (X_0, Θ_0) , requirement map c

Ensure: Plan graph G_P and first feasible plan Π_{first}

```

1: Generate the initial NBA state(s) from  $G_\varphi^-$ 
2: Create the root plan node(s) and initial label(s)  $\lambda_0 = (X_0, \Theta_0, \emptyset)$ 
3: OPEN  $\leftarrow$  stack containing the initial label(s);  $\Pi_{\text{first}} \leftarrow \emptyset$ 
4: while OPEN  $\neq \emptyset$  do
5:   Pop an unexpanded label  $\lambda_i$  attached to  $v_i = (q_{B,i}, \kappa_i)$ 
6:   if the outgoing NBA transitions of  $q_{B,i}$  are not cached then
7:     Generate them on the fly from the outgoing GBA transitions and cache them
8:   end if
9:   Cand  $\leftarrow \emptyset$ 
10:  for all cached  $e_{ij}^B \in \text{Out}(q_{B,i})$  do
11:     $(\text{status}, v_j, \lambda_j, C_{ij}) \leftarrow \text{ExtendLabel}(v_i, \lambda_i, e_{ij}^B, \mathcal{R}, c)$ 
12:    if status = INFEASIBLE then
13:      continue
14:    end if
15:    Add the plan-graph edge  $(v_i, v_j)$  carrying  $(\omega_{ij}, C_{ij})$ 
16:    if  $\lambda_j$  is dominated by an existing label at  $v_j$  then
17:      continue
18:    end if
19:    Insert  $\lambda_j$  at  $v_j$  and store its predecessor label and incoming edge
20:    if status = TERMINAL and  $\Pi_{\text{first}} = \emptyset$  then
21:      Extract  $\Pi_{\text{first}}$  by following the predecessor-label pointers
22:    else if status = FEASIBLE then
23:      Insert  $(\lambda_j, S(\lambda_j))$  into Cand
24:    end if
25:  end for
26:  Sort Cand by nondecreasing score, breaking ties by generation order
27:  Push the sorted labels onto OPEN in reverse order
28: end while
29: Remove nodes and edges that are not on any root-to-terminal path
30: return  $(G_P, \Pi_{\text{first}})$ 

```

Algorithm 3 Execution-label extension

```

1: function ExtendLabel( $v_i, \lambda_i, e_{ij}^B, \mathcal{R}, c$ )
2:   Obtain  $\omega_{ij} = (\gamma_{ij}, \gamma_i^s)$ 
3:   Compute a disjoint synchronized assignment  $C_{ij}$ 
4:   if no feasible assignment exists then
5:     return (INFEASIBLE,  $\emptyset, \emptyset, \emptyset$ )
6:   end if
7:   Compute  $t_{ij}$  and update  $(X_j, \Theta_j, \mathcal{D}_j)$ 
8:   Update  $\kappa_j$  and obtain or create  $v_j = (q_{B,j}, \kappa_j)$ 
9:   Set  $\lambda_j = (X_j, \Theta_j, \mathcal{D}_j)$ 
10:  if  $\kappa_i = q_f \in Q_{B,F}$  and  $q_{B,j} = q_f$  then
11:    return (TERMINAL,  $v_j, \lambda_j, C_{ij}$ )
12:  end if
13:  return (FEASIBLE,  $v_j, \lambda_j, C_{ij}$ )
14: end function

```

D. Graph Simplification and Plan Extraction

After all reachable execution labels have been processed, the plan graph is simplified by standard reachability tests. First, a forward search identifies the nodes reachable from the root. Second, a reverse search from all recorded terminal nodes identifies the nodes from which a terminal continuation is reachable. Only nodes and edges contained in both sets are retained. This operation removes unreachable and nonaccepting dead ends without relying on implementation-specific cut flags or informal sibling-coverage tests.

A feasible plan is extracted from a terminal execution label by repeatedly following its predecessor-label pointer to an initial label. The resulting sequence is reversed to obtain a root-to-terminal path. The subtask and robot assignment associated with each step are read from the corresponding plan-graph edge. Consequently, the extracted task sequence, assignment sequence, robot positions, and completion times are generated by one consistent chain of execution labels. The first such plan is returned during construction, while the completed plan graph is passed to the warm-started branch-and-bound optimizer in Section VII.

7 对 Algorithm 2 和 Algorithm 3 的具体修改清单

7.1 Algorithm 2 必须修改的行

原位置	问题	修改
Require	G_{φ}^{-} 输入后, 正文/Algorithm 1 又调用未剪枝的 G_{φ}	全文统一为 G_{φ}^{-} 。
Line 1	root tuple 含 $T_{\emptyset}, C_{\emptyset}$, 未定义; 若多初始状态也未处理	使用虚拟 root 或每个初始 NBA state 一个 root node; execution label 初始化为 $(X_0, \Theta_0, \emptyset)$ 。
Line 2	“attach root to Bstate” 没有数学对象; 队列与 Q_B 混淆	工作表改为 OPEN, 存 unexpanded execution labels。
Line 5	全局最小 score 与 Section C 的局部 DFS 冲突	改成从 stack 弹出 label; score 只对子 labels 排序。
Lines 6–7	在遍历具体 parent context 前就排序, score 无法计算	先调用 label extension, 得到 successor execution state, 再计算 score。
Line 9	active plan-node anchors 未定义	改成扩展当前 execution label; 同一逻辑节点的多个 labels 各自进入工作表。
Line 10	“Algorithm 3(...)” 不规范	使用函数名 ExtendLabel。
Lines 11–12	representative-predecessor chain 未定义	predecessor pointer 属于 execution label; 计划从 terminal label 提取。
Lines 15–16	只在 NBA state “new” 时入队, 会漏掉后来到达的不同执行状态	是否扩展由 execution label 决定; NBA outgoing transitions 只需缓存一次。
Line 19	只标记 NBA state expanded 与多个执行 context 不兼容	分别标记 execution label 是否已扩展; automaton transition cache 与 label expansion 分开。

7.2 Algorithm 3 必须修改的行

- 删除 $\text{Sig}(T)$ 和 “compatible anchor at $(q'_B, \text{Prog}', \text{Sig}(T))$ ”; 二者没有严格定义, 而且 transition signature 不能支配不同机器人位置/时间。
- 把 assignment 放入 plan edge, 而不是写入共享 node 的唯一 C 字段。
- 先完成精确可行性确认, 再返回 INFEASIBLE。
- terminal condition 必须是 “返回所选 suffix root”, 不能写成笼统的 “completes a prefix-suffix structure”。
- 删除 “update representative payload and propagate to descendants”。若实现确实执行动态 label correction, 需要另写 label-correcting algorithm、终止条件和证明, 不能用一句话带过。

8 可直接替换的关键句子

不建议保留的表达	建议替换
“we retain the on-the-fly construction by a score-prioritized expansion”	“we retain on-the-fly degeneralization and order feasible sibling expansions by a planning score”
“candidate successor Bstates”	“candidate successor NBA states”
“naturally biases the associated plan-graph expansion”	“uses the same local order for the corresponding plan-graph extensions”
“more promising from the planning perspective”	“assigned a smaller remaining-action score”
“runtime payload”	“path-dependent execution label” 或 “predicted robot state”
“compatible anchor”	“execution label at the same logical plan node”
“late-anchor recovery”	“apply the cached outgoing transitions to the newly inserted execution label”
“residual-obligation pressure”	“remaining-action score”
“live corridor”	“the subgraph that is both root-reachable and terminal-reachable”
“hard-dead automaton states or internal cut marks”	从主文删除；若必须保留，先正式定义。
“a child edge can be removed if another sibling covers its successor structure”	用精确的 componentwise dominance 条件替代,或删除。

9 必须同步修改的其他章节

9.1 Definition 5: Subtask

若前一节已经把 symbolic guard 与 alphabet symbol 分开，本节应继续使用

$$e_{ij}^B = (q_{B,i}, \gamma_{ij}, q_{B,j}), \quad \omega_{ij} = (\gamma_{ij}, \gamma_i^s).$$

不要在 Section VI 又改回 σ 。同时统一 $\mathcal{A}(\omega_{ij})$ 的定义：它应表示需要分配机器人的 positive action propositions；负命题和 region propositions 作为路径/holding constraints 处理。

9.2 Section VII: Branch-and-Bound Optimization

当前 BnB 写成 “subtask associated with child node ν_j ”。采用共享图后，应改成 “subtask carried by the plan-graph edge (ν_i, ν_j) ”。此外必须明确 BnB 搜索的是：

- 计划图所保留的 subtask-sequence topology；
- 每条 edge 上所有 feasible robot assignments。

若 plan-graph construction 因某一条贪心 context 不可行而永久删除了 edge, BnB 无法恢复由其他

assignment 才能实现的 continuation。因此，“optimal”最多先写成 *optimal within the retained plan-graph search space*，除非证明 plan graph 保留了所有可行 subtask sequences。

9.3 Complexity Analysis

采用 execution labels 后，复杂度中的 N_P 应解释为“evaluated execution labels”或分别使用 plan nodes 与 labels 的数量。assignment feasibility 若加入 matching fallback，应增加相应项；对于某 robot type 的 m_a 个 action slots 和 R_r 个机器人，标准 bipartite matching 的复杂度可以单独给出，或仅说明 fallback 是多项式时间且只在 greedy failure 时调用。

9.4 Theoretical Analysis

Lemma 4 的证明需要改成以下逻辑链：

1. automaton generation order 不删除 transition；
2. 每个 feasible synchronized subtask 不会因 greedy ordering 而被误删，因为存在 exact feasibility fallback；
3. execution-label dominance 使用 componentwise safe condition；
4. score 只影响顺序，不影响保留集合；
5. terminal condition 精确对应返回同一 suffix root。

如果以上任一条件不成立，就不能继续声称 plan-graph construction 完整。

10 投稿前检查清单

- ☐ 删除 Section VI 开头对 GBA tuple 和同步构造优势的重复说明。
- ☐ 全文删除 *Bstate*，统一为 *NBA state*。
- ☐ 明确真实的 work-list 语义：局部 DFS 或全局 priority queue 二选一。
- ☐ score 在成功扩展具体 execution label 后再计算。
- ☐ 多父节点共享时，subtask 和 assignment 放到 edge。
- ☐ 路径相关位置/时间与逻辑节点分离，或取消节点共享。
- ☐ 删除仅按 current makespan 更新 representative state 的规则。
- ☐ phase 记录具体 suffix root，不再使用通用 suf/dead。
- ☐ greedy failure 经过 exact matching 确认后才能删除 branch。
- ☐ 说明 self-loop guard 如何进入 travel feasibility。
- ☐ 完整定义 J 、已完成 action set 和 remaining-action score。
- ☐ 明确 score 不是 admissible lower bound，也不参与 pruning。
- ☐ 删除 anchor/payload/live corridor/hard-dead/cut mark 等未定义术语。

- plan extraction 从 terminal execution label 沿 predecessor pointers 读取 edge records。
- Section VII、Complexity 和 Theoretical Analysis 与新的图语义同步修改。

11 最终评价

当前 Section VI 的核心思想是明确且有价值的：把任务分配前移到 NBA 生成过程中，并利用首个可行 prefix-suffix 结构为后续优化提供 warm start。真正拖累本节的不是方法缺乏内容，而是数据结构和算法顺序没有被清楚分层。当前写法把共享图的逻辑拓扑、某一路径的机器人执行状态和启发式搜索状态合成一个 plan node，随后又依靠 *anchor*、*representative payload* 和 *late recovery* 补救，导致句子越来越长、术语越来越多，理论保证却越来越难验证。

推荐的“两层表示”只增加一个必要概念——execution label，却可以一次性解决多父节点、路径历史、score 计算、late arrival、plan extraction 和安全 dominance 等问题。完成这一步后，本节可以从目前约三页半压缩到约两页半，同时逻辑会更紧：

on-the-fly NBA transition → plan edge/subtask
→ feasible execution-label extension
→ score-based ordering
→ first terminal label and warm start.

这才是本节应当突出的、读者能够快速理解的技术贡献。